



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 04

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

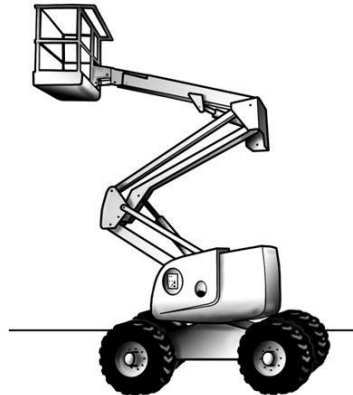
SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 07 de febrero del 2026

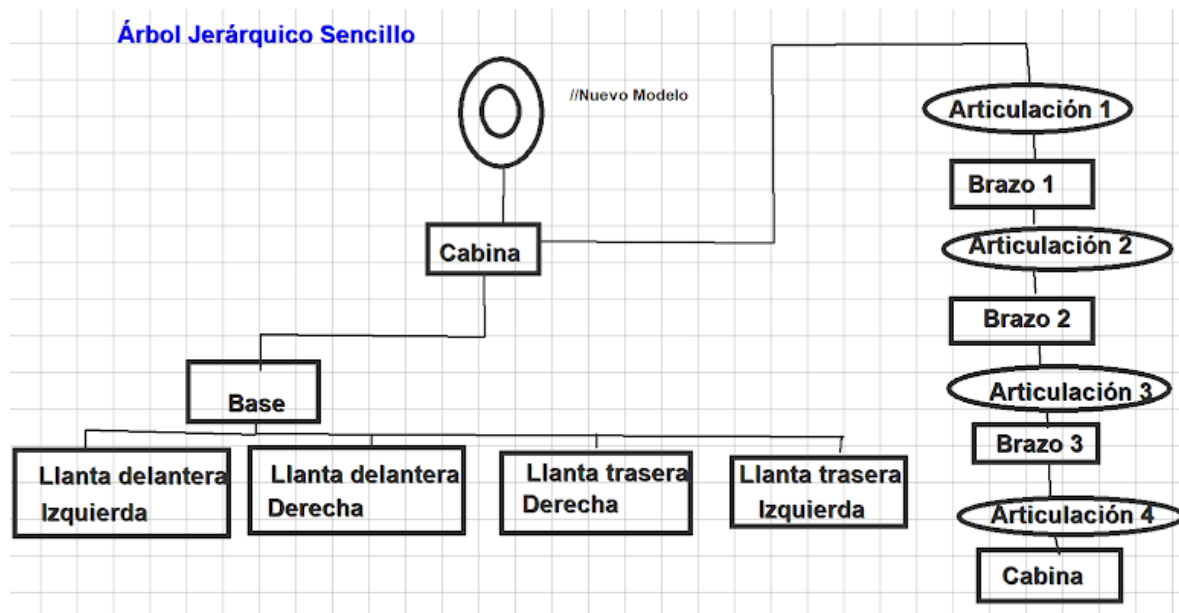
CALIFICACIÓN: _____

DESARROLLO DEL EJERCICIO

Para esta práctica se pidió terminar de construir una grúa funcional con: Cuerpo, brazo de tres partes, cuatro articulaciones y una canasta tal como se muestra en la siguiente figura.



Sin embargo, se decidió terminar la grúa como se pide para la práctica de laboratorio, por ello se documentarán a continuación todos los cambios hechos e instancias colocadas para respetar el modelo jerárquico que se presenta a continuación.



Primeramente, en el main principal se hizo un cambio en la llamada a la función, CrearCilindro(10, 1.0f); subiendo la resolución a 10 para tener una mejor llanta, pero que siga siendo posible apreciar su movimiento.

```
289 // -----
290 // CAMBIO 1: Para tener un mejor cilindro como llanta
291 CrearCilindro(10, 1.0f); //índice 2 en MeshList
292 // -----
```

Pasamos ahora a los archivos de **Window**, tanto el .h como el .cpp siendo que en el primero agregamos los getters para las llantas (se renombraron articulación 5 y 6, mientras que llanta 3 y 4 son getters nuevos).

```
27 // -----
28 // CAMBIO 2: Getters para las llantas
29 GLfloat getLlanta1() { return llanta1; }
30 GLfloat getLlanta2() { return llanta2; }
31 GLfloat getLlanta3() { return llanta3; }
32 GLfloat getLlanta4() { return llanta4; }
33 // -----
```

También se agregaron las nuevas variables.

```
40 // -----
41 // CAMBIO 3: Variables para almacenar articulaciones y llantas
42 GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, llanta1, llanta2, llanta3, llanta4;
43 // -----
```

Ahora, dentro de window.cpp hubo dos cambios cruciales, primero, dentro de `Window::Window(GLint windowHeight, GLint windowWidth)` se eliminó articulación 5 y 6 para colocar e inicializar a llanta 1 a 4.

```
23 // -----
24 // CAMBIO 4: Inicializar variables para las llantas
25 llanta1 = 0.0f;
26 llanta2 = 0.0f;
27 llanta3 = 0.0f;
28 llanta4 = 0.0f;
29 // -----
```

Pero el cambio más crucial fue dentro de `void Window::ManejaTeclado(...)` ya que se agregaron teclas nuevas y limites para el movimiento de los brazos de la grúa obteniendo por resultado la siguiente **TABLA 1**:

TECLA	MOVIMIENTO	LIMITE
F	Brazo 1 hacia adelante	Sup: 10.0f
V	Brazo 1 hacia atrás	Inf: -110.0f
G	Brazo 2 hacia adelante	Sup: 250.0f
B	Brazo 2 hacia atrás	Inf: -60.0f
H	Brazo 3 hacia adelante	Sup: 65.0f
N	Brazo 3 hacia atrás	Inf: -220.0f
J	Canasta hacia adelante	Sup: 180.0f
M	Canasta hacia atrás	Inf: -2.0f

K	Llanta 1 hacia adelante	-
L	Llanta 2 hacia adelante	-
I	Llanta 3 hacia adelante	-
O	Llanta 4 hacia adelante	-

```

135 // ===== CAMBIO 5: Controles de grua =====
136 // =====
137
138 // 1.- BRAZO ARTICULADO: ARTICULACIÓN 1
139 if (key == GLFW_KEY_F){ // Tecla F: Movimiento hacia adelante
140     theWindow->articulacion1 += 5.0f;
141     if (theWindow->articulacion1 > 10.0f){ // LimSup
142         theWindow->articulacion1 = 10.0f;
143     }
144 if (key == GLFW_KEY_V){ // Tecla V: Movimiento hacia atrás
145     theWindow->articulacion1 -= 5.0f;
146     if (theWindow->articulacion1 < -110.0f){ // LimInf
147         theWindow->articulacion1 = -110.0f;
148     }
149
150 // 1.- BRAZO ARTICULADO: ARTICULACIÓN 2
151 if (key == GLFW_KEY_G){ // Tecla G: Hacia adelante
152     theWindow->articulacion2 += 5.0f;
153     if (theWindow->articulacion2 > 250.0f){
154         theWindow->articulacion2 = 250.0f;
155     }
156 if (key == GLFW_KEY_B){ // Tecla B: Hacia atrás
157     theWindow->articulacion2 -= 5.0f;
158     if (theWindow->articulacion2 < -60.0f){ // LimInf
159         theWindow->articulacion2 = -60.0f;
160     }
161
162 // 1.- BRAZO ARTICULADO: ARTICULACIÓN 3
163 if (key == GLFW_KEY_H){ // Tecla H: Hacia adelante
164     theWindow->articulacion3 += 5.0f;
165     if (theWindow->articulacion3 > 65.0f){ // LimSup
166         theWindow->articulacion3 = 65.0f;
167     }
168 if (key == GLFW_KEY_N){ // Tecla N: Hacia atrás
169     theWindow->articulacion3 -= 5.0f;
170     if (theWindow->articulacion3 < -220.0f){ // LimInf
171         theWindow->articulacion3 = -220.0f;
172     }
173
174 // 1.- BRAZO ARTICULADO: ARTICULACIÓN 4
175 if (key == GLFW_KEY_J){ // Tecla J: Hacia adelante
176     theWindow->articulacion4 += 5.0f;
177     if (theWindow->articulacion4 > 180.0f){ // LimSup
178         theWindow->articulacion4 = 180.0f;
179     }
180 if (key == GLFW_KEY_M){ // Tecla M: Hacia atrás
181     theWindow->articulacion4 -= 5.0f;
182     if (theWindow->articulacion4 < -2.0f){ // LimInf
183         theWindow->articulacion4 = -2.0f;
184     }
185
186 // =====
187
188 // 2.- CILINDROS: Llanta 1 (Delantera Izquierda)
189 if (key == GLFW_KEY_K){
190     theWindow->llanta1 += 5.0f;
191 // 2.- CILINDROS: Llanta 2 (Delantera Derecha)
192 if (key == GLFW_KEY_L){
193     theWindow->llanta2 += 5.0f;
194 // 2.- CILINDROS: Llanta 3 (Trasera Izquierda)
195 if (key == GLFW_KEY_I){
196     theWindow->llanta3 += 5.0f;
197 // 2.- CILINDROS: Llanta 4 (Trasera Derecha)
198 if (key == GLFW_KEY_O){
199     theWindow->llanta4 += 5.0f;
200 // =====

```

Regresando a nuestro main, dentro del ciclo `while (!mainWindow.getShouldClose())` comenzamos con el modelado jerárquico de la grúa, primero establecemos nuestro nodo raíz/padre como la cabina (prisma rectangular). Es importante mencionar que aquí SI inicializamos la matriz de transformación principal, y es gracias a la variable `modelaux` que podemos aplicar transformaciones, escalamientos y rotaciones jerárquicas para que los demás componentes se conecten a nuestra cabina heredando/siendo hijos de su posición global y rotación sin deformarse.

```

353 // *****
354 // ***** MODELADO JERÁRQUICO: GRUA *****
355 // *****
356
357 // =====
358 // CAMBIO 6: NODO RAÍZ: CABINA
359 // =====
360 model = glm::mat4(1.0f);
361 model = glm::translate(model, glm::vec3(0.0f, 3.0f, -4.0f));
362 modelaux = model; // GUARDAMOS: Centro de la cabina (padre)
363 model = glm::scale(model, glm::vec3(4.0f, 2.0f, 3.0f));
364 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
365 color = glm::vec3(0.859f, 0.533f, 0.318f); // Naranja base
366 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
367 meshList[0]->RenderMesh();

```

Teniendo la base, vamos a crear la base de la grúa con una pirámide cuadrangular, esta hereda de la posición de la cabina con `modelaux`. También creamos un modelo de apoyo llamado `modelBaseCab` para que de ahí colguemos nuestras cuatro llantas, siendo que a cada una se le aplica una rotación de 90° base, además de la rotación con teclado.

```

369 // -----
370 // CAMBIO 7: BASE Y LLANTAS (Heredan de la Cabina)
371 // -----
372 // ---- BASE PIRÁMIDE (Hijo de Cabina)
373 model = modelaux;
374 model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f)); // Bajamos la base
375 glm::mat4 modelBaseCab = model; // GUARDAMOS: Centro de la base (padre de llantas)
376 model = glm::scale(model, glm::vec3(3.0f, 1.5f, 4.0f));
377 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
378 color = glm::vec3(0.3f, 0.3f, 0.3f); // Gris oscuro
379 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
380 meshList[4]->RenderMeshGeometry();
381
382 // ---- LLANTAS (Hijas de la Base)
383 glm::vec3 escalaLlanta = glm::vec3(0.8f, 0.8f, 1.0f);
384 glm::vec3 colorLlanta = glm::vec3(0.188f, 0.188f, 0.184f);
385 // Llanta 1: Delantera Izquierda (Tecla K)
386 model = modelBaseCab;
387 model = glm::translate(model, glm::vec3(-1.0f, -0.5f, 2.3f));
388 model = glm::rotate(model, glm::radians(mainWindow.getLlanta1()), glm::vec3(0.0f, 0.0f, 1.0f));
389 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
390 model = glm::scale(model, escalaLlanta);
391 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
392 glUniform3fv(uniformColor, 1, glm::value_ptr(colorLlanta));
393 meshList[2]->RenderMeshGeometry();
394 // Llanta 2: Delantera Derecha (Tecla L)
395 model = modelBaseCab;
396 model = glm::translate(model, glm::vec3(1.0f, -0.5f, 2.3f));
397 model = glm::rotate(model, glm::radians(mainWindow.getLlanta2()), glm::vec3(0.0f, 0.0f, 1.0f));
398 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
399 model = glm::scale(model, escalaLlanta);
400 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
401 glUniform3fv(uniformColor, 1, glm::value_ptr(colorLlanta));
402 meshList[2]->RenderMeshGeometry();

```

```

403 // Llanta 3: Trasera Izquierda (Tecla I)
404 model = modelBaseCab;
405 model = glm::translate(model, glm::vec3(-1.0f, -0.5f, -2.3f));
406 model = glm::rotate(model, glm::radians(mainWindow.getLlanta3()), glm::vec3(0.0f, 0.0f, 1.0f));
407 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
408 model = glm::scale(model, escalaLlanta);
409 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
410 glUniform3fv(uniformColor, 1, glm::value_ptr(colorLlanta));
411 meshList[2]->RenderMeshGeometry();
412 // Llanta 4: Trasera Derecha (Tecla O)
413 model = modelBaseCab;
414 model = glm::translate(model, glm::vec3(1.0f, -0.5f, -2.3f));
415 model = glm::rotate(model, glm::radians(mainWindow.getLlanta4()), glm::vec3(0.0f, 0.0f, 1.0f));
416 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
417 model = glm::scale(model, escalaLlanta);
418 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
419 glUniform3fv(uniformColor, 1, glm::value_ptr(colorLlanta));
420 meshList[2]->RenderMeshGeometry();

```

Un cambio extra que se realizo fue poner pequeños detalles en la grúa, tales como 3 ventanas (cubos aplanados), un contrapeso que suele tener la grúa y un motor con un cilindro en la parte delantera, para ello se instanciaron 5 modelos extra reutilizando modelaux de la cabina.

```

423 // -----
424 // CAMBIO 8: DETALLES DE LA CABINA (Heredan de la Cabina)
425 // -----
426 glm::vec3 colorVentana = glm::vec3(0.725f, 0.863f, 0.922f); // Azul vidrio
427 // ---- CONTRAPESO TRASERO
428 model = modelaux;
429 model = glm::translate(model, glm::vec3(2.5f, -0.5f, 0.0f));
430 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 2.8f));
431 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
432 color = glm::vec3(0.388f, 0.345f, 0.314f);
433 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
434 meshList[0]->RenderMesh();
435 // ---- VENTANA FRONTAL
436 model = modelaux;
437 model = glm::translate(model, glm::vec3(-2.0f, 0.2f, 0.0f));
438 model = glm::scale(model, glm::vec3(0.1f, 1.0f, 2.0f));
439 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
440 color = glm::vec3(colorVentana);
441 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
442 meshList[0]->RenderMesh();
443 // ---- VENTANA DERECHA
444 model = modelaux;
445 model = glm::translate(model, glm::vec3(0.0f, 0.2f, 1.51f));
446 model = glm::scale(model, glm::vec3(3.5f, 1.0f, 0.1f));
447 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
448 color = glm::vec3(colorVentana);
449 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
450 meshList[0]->RenderMesh();
451 // ---- VENTANA IZQUIERDA
452 model = modelaux;
453 model = glm::translate(model, glm::vec3(0.0f, 0.2f, -1.51f));
454 model = glm::scale(model, glm::vec3(3.5f, 1.0f, 0.1f));
455 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
456 color = glm::vec3(colorVentana);
457 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
458 meshList[0]->RenderMesh();
459 // ---- CILINDRO FRONTAL (Motor)
460 model = modelaux;
461 model = glm::translate(model, glm::vec3(-2.0f, -0.7f, 0.0f)); // Justo debajo de la ventana
462 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
463 model = glm::scale(model, glm::vec3(0.6f, 3.0f, 0.5f));
464 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
465 color = glm::vec3(0.859f, 0.533f, 0.318f); // Mismo color naranja de la cabina
466 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
467 meshList[2]->RenderMeshGeometry();

```


Finalmente, a partir de los dos brazos de la grúa ya programados por el profesor, se buscó instanciar el tercer brazo faltante junto a su canasta. Comenzando desde la cabina, se hicieron de manera secuencial tres brazos conectados por esferas que actúan como pivotes (articulaciones). En cada paso, la pieza actual guarda su matriz de transformación, pero sin escalar y se la heredamos a la siguiente pieza. Esto incluye la parte final (la canasta), garantizando así que nuestras rotaciones aplicadas en las articulaciones base afecten a todos los nodos hijos que sigan, imitando la física real de una grúa articulada.

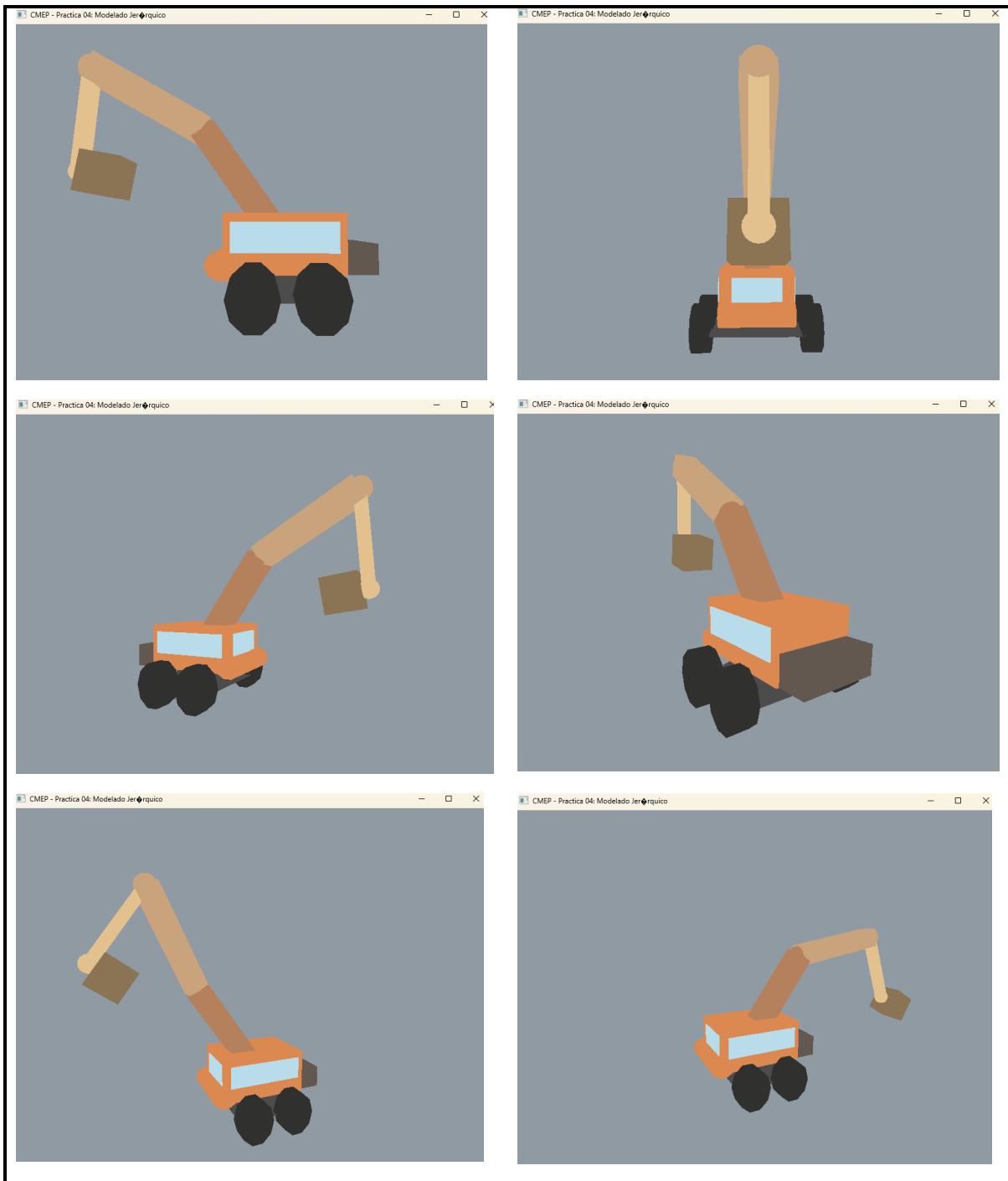
```

470 // -----
471 // CAMBIO 9: BRAZO ARTICULADO (Hijos)
472 // -----
473 // ---- ARTICULACIÓN 1 Y BRAZO 1 (Hijo de Cabina)
474 model = modelaux; // Partimos nuevamente desde la cabina
475 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
476 model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
477 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
478 modelaux = model; // GUARDAMOS: Centro Brazo 1
479 model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
480 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
481 color = glm::vec3(0.710f, 0.502f, 0.361f);
482 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
483 meshList[0] -> RenderMesh();
484 // ---- ARTICULACIÓN 2 (Hijo de Brazo 1)
485 model = modelaux;
486 model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
487 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
488 modelaux = model; // GUARDAMOS: Pivote Articulación 2
489 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
490 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
491 sp.render();
492 // ---- BRAZO 2 (Hijo de Articulación 2)
493 model = modelaux;
494 model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
495 modelaux = model; // GUARDAMOS: Centro Brazo 2
496 model = glm::scale(model, glm::vec3(1.0f, 5.0f, 1.0f));
497 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
498 color = glm::vec3(0.788f, 0.639f, 0.482f);
499 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
500 meshList[0] -> RenderMesh();
501 // ---- ARTICULACIÓN 3 (Hijo de Brazo 2)
502 model = modelaux;
503 model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
504 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
505 modelaux = model; // GUARDAMOS: Pivote Articulación 3
506 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
507 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
508 sp.render();
509 // ---- BRAZO 3 (Hijo de Articulación 3)
510 model = modelaux;
511 model = glm::translate(model, glm::vec3(2.0f, 0.0f, 0.0f));
512 modelaux = model; // GUARDAMOS: Centro Brazo 3
513 model = glm::scale(model, glm::vec3(4.0f, 0.5f, 0.5f));
514 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
515 color = glm::vec3(0.890f, 0.757f, 0.561f);
516 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
517 meshList[0] -> RenderMesh();
518 // ---- ARTICULACIÓN 4 (Hijo de Brazo 3)
519 model = modelaux;
520 model = glm::translate(model, glm::vec3(2.0f, 0.0f, 0.0f));
521 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
522 modelaux = model; // GUARDAMOS: Pivote Articulación 4
523 model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
524 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
525 sp.render();
526 // ---- CANASTA (Hijo de Articulación 4)
527 model = modelaux;
528 model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
529 modelaux = model; // GUARDAMOS: Centro Canasta
530 model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
531 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
532 color = glm::vec3(0.541f, 0.455f, 0.325f);
533 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
534 meshList[0] -> RenderMesh();

```

EJECUCIÓN

Para la ejecución se usa la cámara libre y las teclas WASD (para la cámara) mientras que las teclas F,V,G,B,H,N,J,M,I,O,K y L se usan para las rotaciones (*Ver tabla 1*)



Se adjunta link a drive del GIF para apreciar mejor el movimiento

https://drive.google.com/file/d/1Jd96SftVYo3eJRBqTCvwUgHqk_6mKyh2/view?usp=sharing

PROBLEMAS PRESENTADOS

Para esta práctica no hubo problemas de ejecución presentados pero si dificultades para la herencia de modelos, en algunas ocasiones se heredaban las transformaciones o rotaciones por no colocar en la línea adecuada el modelAux.

CONCLUSIONES

Sobre los ejercicios de clase

Para este ejercicio realizar la implementación de una grúa articulada demostró de manera práctica la importancia del modelado jerárquico en nuestra materia, comprobamos así que estructurar un modelo mediante un árbol de dependencias (Padre-Hijo) optimiza bastante el entendimiento y posicionamiento de elementos con varios movimientos. Al definir la cabina como nodo raíz/padre, cualquier transformación global afectó automáticamente a la base, las llantas y el brazo articulado en tres partes, manteniendo un modelo que de poco en poco podemos acercarlo a la realidad.

Comentarios generales

El tener dos códigos main desde inicio de la práctica facilitó mucho la sesión, aunque se intentaba seguir el ritmo del profesor ayudado en gran medida tener un código ya preparado con lo que se vio en la sesión, así se pudo prestar más atención a la explicación de clase.